

Algorithmen und Datenstrukturen

Übung 8: Sortieren III

Robert

2026-07-03

Wiederholung Heap Sort

Heap Sort

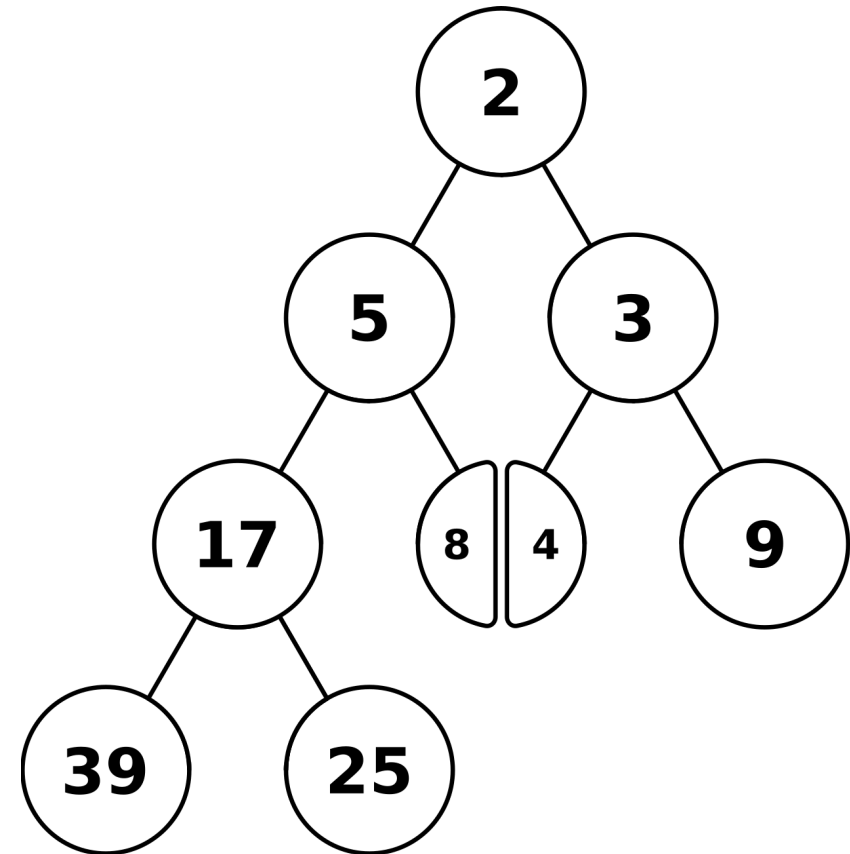
- Heap Sort sortiert immer das größte Element nach hinten
- gleichzeitig verschiebt es das nächst kleinere Element ganz nach vorne ins Array, damit es nicht mehr danach suchen muss
- die Idee von HeapSort ist, dass es wie SelectionSort funktioniert, aber statt einer Komplexität von $O(n^2)$ hat es eine Komplexität von $O(n \cdot \log(n))$ hat.

Ablauf

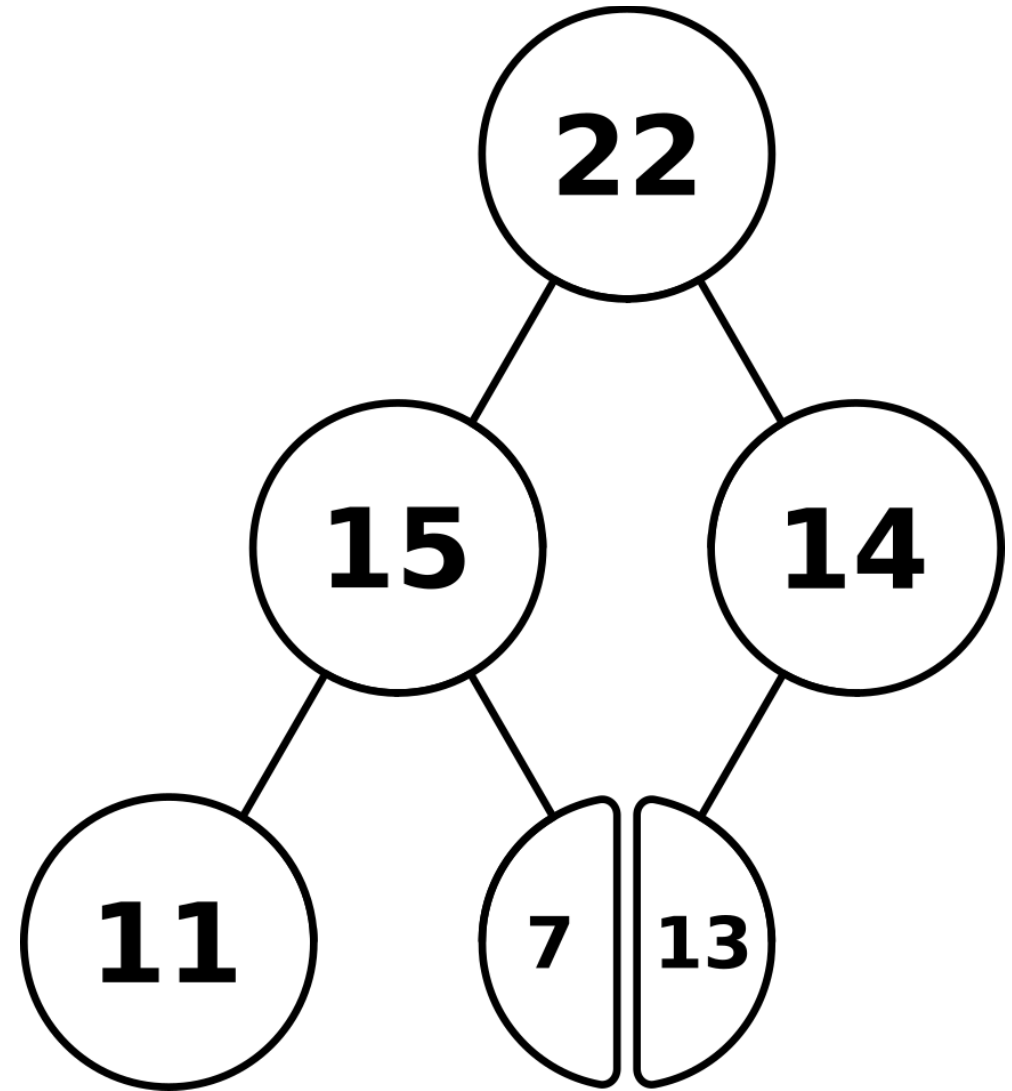
1. Binärbaum: das Array wird in einen linksvollständigen, binären Baum umgewandelt
 - das erste Element bildet die Wurzel und die nachfolgenden Elemente werden Ebene für Ebene eingefügt
 - linksvollständig: jede Ebene wird von links nach rechts aufgefüllt
2. Heapaufbau: die Max-Heap-Eigenschaft wird im Baum hergestellt
3. Sortierphase:
 - das Element in der Wurzel wird einsortiert und verschwindet aus dem Baum
 - das erste Element (das größte Element) im Array wird mit dem letzten Element vertauscht
 - danach muss die Max-Heap-Eigenschaft wieder hergestellt werden

Heap-Eigenschaft

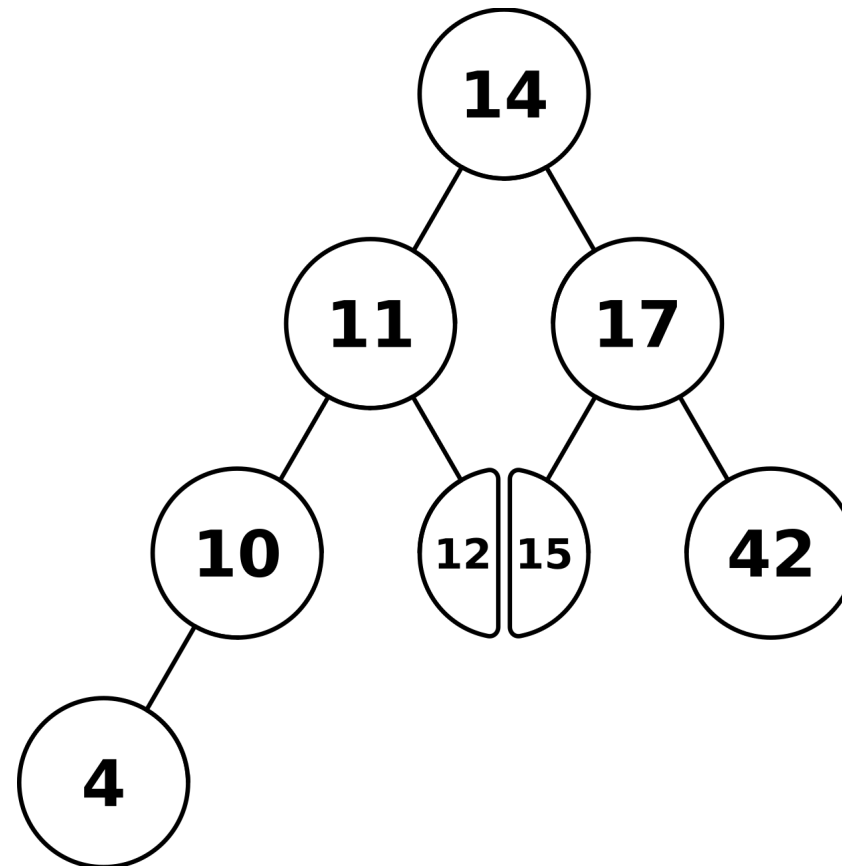
- die Heap-Eigenschaft beschreibt, wo sich die Elemente in einem Binärbaum abhängig von ihrem Wert befinden
- Min-Heap
 - der Schlüssel eines Knoten muss kleiner sein, als der Schlüssel des Knotens, der darunter liegt
 - Beispiel-Array:
[2, 5, 3, 17, 8, 4, 9, 39, 25]



- Max-Heap
 - der Schlüssel eines Knotens muss größer sein, als der Schlüssel des Knotens, der darunter liegt
 - Beispiel-Array:
[22, 15, 14, 11, 7, 13]



- Unterschied zu einem Suchbaum:
 - im linken Teilbaum befinden sich alle Element, die kleiner sind als der Schlüssel
 - im rechten Teilbaum befinden sich alle Elemente, die größer sind als der Schlüssel



Komplexität

- der Aufbau des Heaps hat eine Komplexität von $O(n)$
 - die Wurzel könnte maximal um $\log_2 n (= h)$ Ebenen sinken (wenn in der Wurzel das kleinste Element ist), wodurch die Komplexität eigentlich bei $O(n \cdot \log(n))$ liegt
 - aber weil die meisten Knoten unten im Baum sind, können nur die wenigen Knoten oben im Baum um $\log_2(n)$ Ebenen nach unten wandern
 - deshalb reicht eine Komplexität von $O(n)$ schon aus
- die Sortierphase hat eine Komplexität von $O(n \cdot \log(n))$
 - jedes der n Elemente kommt einmal ganz nach vorne ins Array (in die Wurzel)
→ $O(n)$
 - der Algorithmus senkt dann immer das größte Element maximal $\log_2 n$ Schritte im Binärbaum ab
 - also insgesamt $O(n \cdot \log(n))$ Operationen
- insgesamt hat HeapSort also eine Komplexität von $O(n \cdot \log(n))$

Vorsortierung

- die Komplexität ist unabhängig von der Vorsortierung
 - selbst wenn das Array bereits sortiert ist, braucht HeapSort $O(n \cdot \log(n))$ Operationen
 - es führt immer erstmal den Heapaufbau durch, weshalb die Vorsortierung durcheinander gebracht wird

Aufgabe 1

1 a)

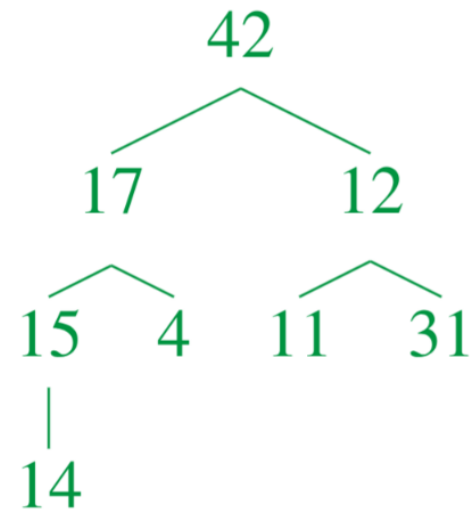
Welche Elementpaare verletzen die Max-Heap-Eigenschaft?

Welche alternativen Werte erfüllen die Heap-Eigenschaft?

$$A = [42, 17, 12, 15, 4, 11, 31, 14]$$

- als Binärbaum:

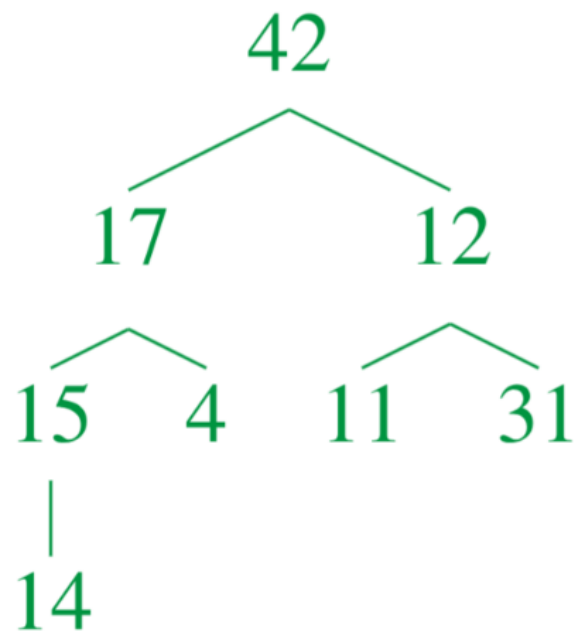
- das Paar 12-31 verletzt die Max-Heap-Eigenschaft
- alternative Werte:
 - statt 12 $\rightarrow 31 < x < 42$
 - oder statt 31 $\rightarrow x < 12$



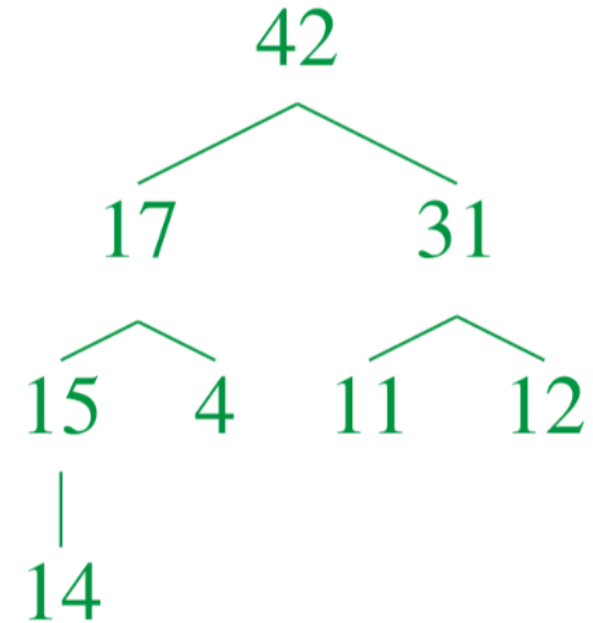
1 b)

Sortiere das Array A mit HeapSort. Gebe die Zwischenschritte als Binärbaum an.

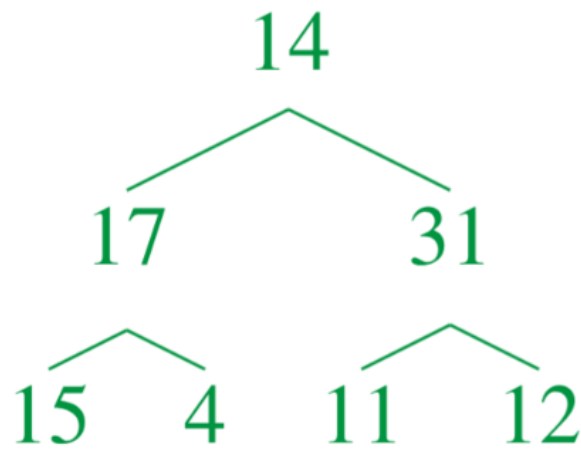
$$A = [42, 17, 12, 15, 4, 11, 31, 14]$$



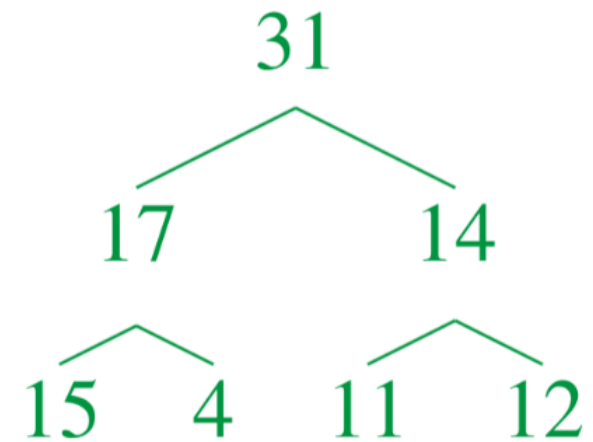
heapify $\Rightarrow$



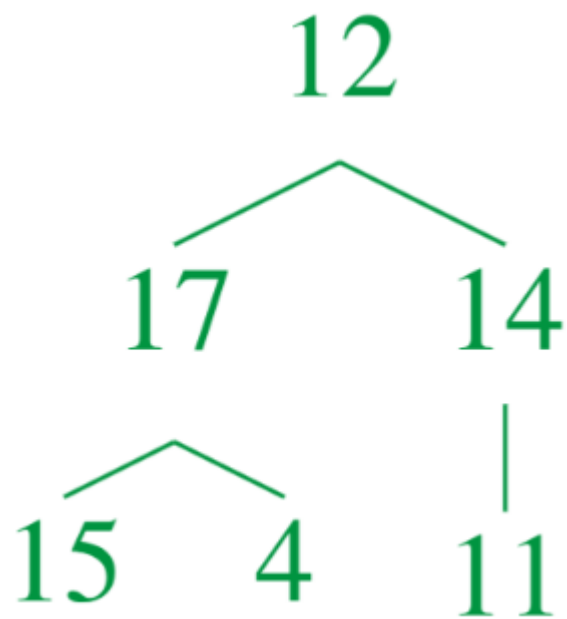
- $[42, 17, 31, 15, 4, 11, 12, 14]$
- 42 und 14 vertauschen
 $\Rightarrow A = [14, 17, 31, 15, 4, 11, 12, (42)]$ (die Klammern markieren den sortierten Teil)



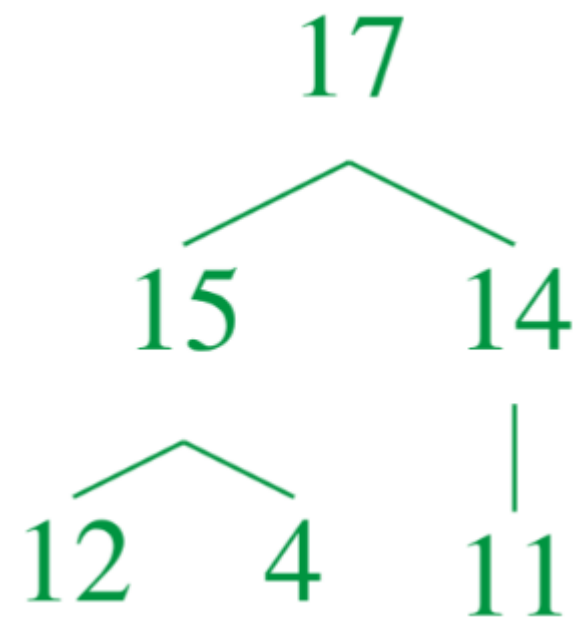
sink $\Rightarrow$



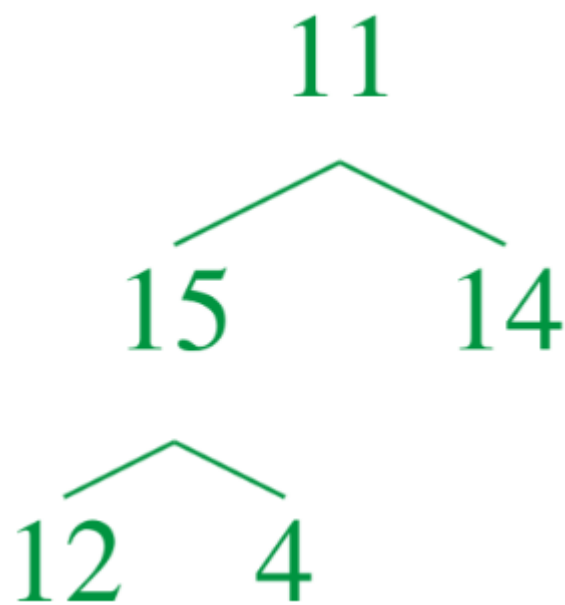
- $[31, 17, 14, 15, 4, 11, 12, (42)]$
- 12 und 31 vertauschen
 $\Rightarrow A = [12, 17, 14, 15, 4, 11, (31, 42)]$



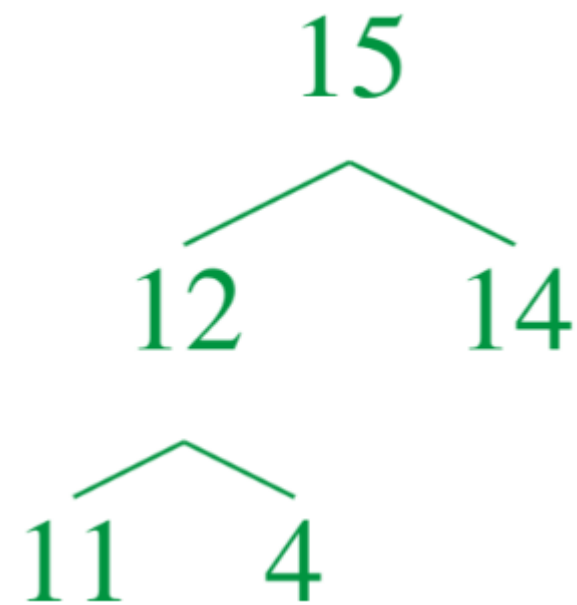
sink $\Rightarrow$



- $[17, 15, 14, 12, 4, 11, (31, 42)]$
- 11 und 17 vertauschen
 $\Rightarrow A = [11, 15, 14, 12, 4, (17, 31, 42)]$



sink $\Rightarrow$



- $[15, 12, 14, 11, 4, (17, 31, 42)]$
- 15 und 4 vertauschen
 $\Rightarrow A = [4, 12, 14, 11, (15, 17, 31, 42)]$



sink $\Rightarrow$



- $[14, 12, 4, 11, (15, 17, 31, 42)]$
- 14 und 11 vertauschen
 $\Rightarrow A = [11, 12, 4, (14, 15, 17, 31, 42)]$



- $[12, 11, 4, (14, 15, 17, 31, 42)]$
- 12 und 4 vertauschen
 $\Rightarrow A = [4, 11, (12, 14, 15, 17, 31, 42)]$

4
|
11

sink
⇒

11
|
4

- $[11, 4, (12, 14, 15, 17, 31, 42)]$
- 11 und 4 vertauschen
⇒ $A = [4, 11, 12, 14, 15, 17, 31, 42]$

Wiederholung TimSort

TimSort

TimSort ist ein Sortieralgorithmus, der ein Hybrid aus MergeSort und InsertionSort ist.

- InsertionSort: Geht das Array von vorne nach hinten durch und fügt die Element an der richtigen Stelle ein
- MergeSort: Teilt Array in Teilfolgen auf, bis einzelne Elemente übrigbleiben und fügt sie dann in der richtigen Reihenfolge wieder zusammen
- TimSort:
 1. durchläuft das Array und identifiziert aufsteigende und absteigende Folgen von Elementen (runs), die eine Mindestlänge (**minrun**) haben
 2. mit InsertionSort werden Folgen von Elementen vergrößert, deren Länge kleiner ist, als **minrun**, indem Elemente hinzugefügt werden, sodass am Ende nur Teilfolgen mit einer Mindestlänge von **minrun** übrigbleiben
 3. mit MergeSort werden die Teilfolgen zu einer ganzen Folge zusammengefügt
- Visualisierung: <https://www.geeksforgeeks.org/dsa/timsort/>

Aufgabe 2

2 a)

Ist es sinnvoll, als ersten Schritt die runs zu identifizieren?

- ja, es ist sinnvoll, da in der Praxis komplett ungeordnete Daten meist selten vorkommen
- außerdem kann das Sortieren beschleunigt werden, wenn bekannt ist, ob es absteigende runs gibt

2 b)

Was ist die Laufzeit im Best- und Worst-Case von TimSort?

- Best-Case:
 - Array ist bereits sortiert $\rightarrow O(n)$
 - beim ersten Durchlauf bildet das ganze Array einen run

- Worst-Case:
 - die MergeSort-Phase hat eine Laufzeit von $O(n \cdot \log(n))$
 - die InsertionSort-Phase hat eine Laufzeit von $O(n)$ (kann also vernachlässigt werden)
 - da InsertionSort nur Folgen behandelt, die kleiner als `minrun` sind, kann es maximal $\frac{n}{\text{minrun}}$ Teilfolgen geben
 - die Laufzeit beim Worst-Case für InsertionSort bei einer Teilfolge der Länge `minrun` liegt bei $O(\text{minrun}^2)$
 - das heißt, $\frac{n}{\text{minrun}} \cdot O(\text{minrun}^2) = O(n \cdot \text{minrun})$ und weil `minrun` eine Konstante ist, hat die InsertionSort-Phase eine Laufzeit von $O(n)$
 - $O(n) + O(n \cdot \log(n)) = O(n \cdot \log(n))$
 - Beispiel: [4, 3, 2, 1, 8, 7, 6, 5, 12, 11, 10, 9, 16, 15, 14, 13]

Aufgabe 3

CountingSort

- CountingSort ist ein Sortieralgorithmus, der ohne Vergleiche auskommt
 - er ist sehr effizient, wenn die Wertemenge relativ klein ist, verglichen mit der Anzahl der Elemente
- Vorgehen:
 - er zählt, wie oft ein Wert vorkommt
 - und platziert die Elemente, die diesen Wert haben, dann an die entsprechende Stelle im Outputarray

Beispiel:

- nach einem Test werden die Schüler nach der Häufigkeit der Noten sortiert
- [Timmy: 2, Tommy: 3, Lilly: 1, Willi:4, Molly: 3, Lotte: 4, Jimmy: 2, Elli: 4, Karl-Heinz: 6]
- sortiert mit CountingSort:
 - $\Rightarrow$ [Lilly, Karl-Heinz, Timmy; Jimmy, Tommy, Molly, Willi, Lotte, Elli]

Sortiere die Bücher anhand ihrer Kategorien mit CountingSort.

Name	Kategorie	Name	Kategorie
A	Kinderbuch	F	Science-Fiction
B	Fantasy	G	Science-Fiction
C	Kinderbuch	H	Geschichtsbuch
D	Geschichtsbuch	I	Science-Fiction
E	Science-Fiction	J	Kinderbuch

1. Wie oft kommt jede Kategorie vor?

- Kinderbuch: 3 (A, C, J)
- Fantasy: 1 (B)
- Geschichtsbuch: 2 (D, H)
- Science-Fiction: 4 (E, F, G, I)

2. Die Bücher anhand der lexikographischen Ordnung ihrer Kategorie geordnet:

$\Rightarrow [B, D, H, A, C, J, E, F, G, I]$

Aufgabe 4

SuffixArray

- ein SuffixArray stellt alle Suffixe eines Strings dar
 - ein Suffix eines Strings ist ein Teilstring, der am Ende des Strings beginnt und bis zum Anfang reicht
 - Beispiel: Banane $\rightarrow$ [Banane, anane, nane, ane, ne, e]
 - die Suffixe werden dann lexikographisch sortiert
 - [anane, ane, Banane, e, nane, ne]
 - und die ursprünglichen Indizes der sortierten Suffixe werden zurückgegeben:
 - [1, 3, 0, 5, 2, 4]

4 a)

Erstelle das SuffixArray zu $S = \text{SUFFIX\$}$

- Was sind die ganzen Suffixe und ihre Indizes?

Index	0	1	2	3	4	5	6
	S	U	F	F	I	X	\$
	U	F	F	I	X	\$	
	F	F	I	X	\$		
	F	I	X	\$			
	I	X	\$				
	X	\$					
	\$						

- alle Suffixe lexikographisch geordnet:

Position	0	1	2	3	4	5	6
Index	6	2	3	4	0	1	5
	\$	F	F	I	S	U	X
		F	I	X	U	F	\$
		I	X	\$	F	F	
		X	\$		F	I	
		\$			I	X	
					X	\$	
					\$		

- SuffixArray: [6, 2, 3, 4, 0, 1, 5]

4 b)

Suche $T = FIX$ im SuffixArray mit binärer Suche.

Position Index	0	1	2	3	4	5	6
	6	2	3	4	0	1	5
	\$	F	F	I	S	U	X
		F	I	X	U	F	\$
		I	X	\$	F	F	
		X	\$		F	I	
		\$			I	X	
					X	\$	
					\$		

- das Array insgesamt 7 Positionen und die binäre Suche fängt in der Mitte an:
 - $\frac{0+6}{2} = 3 \rightarrow$ Position 3 vergleichen:
 - $FIX < IX\$$ (weil F im Alphabet vor dem I kommt)
 - es wird in der linke Hälfte weitergesucht
 - $\frac{0+2}{2} = 1 \rightarrow$ Position 1 vergleichen:
 - $FIX > FFIX\$$
 - es wird in der rechten Hälfte weitergesucht
 - $\frac{2+2}{2} = 2 \rightarrow$ Position 2 vergleichen:
 - $FIX = FIX\$ \rightarrow$ Suffix gefunden
 - Index von $FIX\$$ ist 3